

About PageRank Vector

By 3611 이준규 | 선형대수학

참고 문헌

Page, L., Brin, S., Motwani, R., & Winograd, T. (1998). "The PageRank Citation Ranking: Bringing Order to the Web." *Stanford InfoLab Technical Report*.

Brin, S. & Page, L. (1998). "The Anatomy of a Large-Scale Hypertextual Web Search Engine." *Computer Networks and ISDN Systems*, 30, 107–117.

Langville, A. & Meyer, C. (2004). "Deeper Inside PageRank." *Internet Mathematics*, Vol. 1, No. 3, 335–380.

Cornell University MEC (2009). "PageRank Algorithm — The Mathematics of Google Search." <https://pi.math.cornell.edu/~mec/Winter2009/RalucaRemus/Lecture3/lecture3.html>

Anton, H. & Busby, R. (2003). *Contemporary Linear Algebra*. Wiley.

목차

참고 문헌

문제 정의

주제 설명

행렬을 통한 모델링

정상 상태(Steady State)와 거듭제곱법

Dead End 문제

Spider Trap 문제

결론 — 구글 행렬

구현 코드(GitHub)와 수렴 판정

문제 정의

기존 브라우저에서 검색을 할 때

상단에 어떤 페이지를 노출할지 결정하는 것은 검색 키워드의 빈도수였다

문제 정의

기존 브라우저에서 검색을 할 때

상단에 어떤 페이지를 노출할지 결정하는 것은 검색 키워드의 빈도수였다

그러나 이것은 쉽게 조작이 가능했기 때문에

다른 결정 기준이 필요했다

주제 설명

Page와 Brin은 다른 방법을 제안했다

다른 페이지들로부터 링크(인용)를 많이 받는 페이지일수록 중요한 것이다

주제 설명

Page와 Brin은 다른 방법을 제안했다

다른 페이지들로부터 링크(인용)를 많이 받는 페이지일수록 중요한 것이다

그렇게 등장한 것이 PageRank

주제 설명

그리고 PageRank의 문제와 모호함을 보완해 만든 것이 **구글 행렬**이다



행렬을 통한 모델링

벡터를 구성한다

벡터 $v \in \mathbb{R}^n$ 는 n 개 페이지의 중요도를 벡터로 정리한 것이다

$$v = \begin{pmatrix} v_a \\ v_b \\ v_c \\ v_d \end{pmatrix}$$

원리적으로 벡터를 구성하는 원소들의 합이 1일 필요는 없다

그러나 초기 벡터의 원소들 합이 1이 되도록 한다

이는 뒤에서 다루게 될 것이다

행렬을 통한 모델링

그리고 이 벡터를 행렬을 통해 변환할 것이다

PageRank 벡터의 원소 중요도를

마치 그 페이지를 방문할 통계적 확률과 같다고 해석해도 좋을 것 같다

$$M_{ij} = \begin{cases} \frac{1}{|\text{out}(j)|} & \text{page } j \text{ links to page } i \\ 0 & \text{else} \end{cases}$$

여기서 $|\text{out}(j)|$ 는 페이지 j 의 나가는 링크 수이다

페이지 j 가 가진 중요도를 자신이 링크하는 페이지에 균등하게 분배한다

행렬을 통한 모델링

이렇게 정의하면 나가는 링크가 있는 모든 열의 합이 1 이 된다

이런 행렬을 열 확률 행렬 (Column-Stochastic Matrix)이라 한다

행렬을 통한 모델링

예시: $A \rightarrow B / A \rightarrow C / B \rightarrow C / C \rightarrow A / D$ 는 링크 없음

$$M = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0.5 & 0 & 0 & 0 \\ 0.5 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

나가는 링크가 있는 **A, B, C**는 열의 합에 1이다

그러나 나가는 링크가 없는 **D**는 합이 0이다

이 문제는 이후에 뒤에서 다룰 것이다

행렬을 통한 모델링

열의 합이 모두 1인 확률 행렬은 항상 고유값 $\lambda = 1$ 을 가진다.

행렬을 통한 모델링

증명

행벡터 $\mathbf{1}^T = (1, 1, \dots, 1)$ 에 대해 열의 합이 모두 1이므로 $\mathbf{1}^T M = \mathbf{1}^T$

이는 1이 M^T 의 고유값 1에 대응하는 고유벡터임을 의미하고
따라서 M 의 고유값 중 1이 존재한다

또한 모든 확률값이 $[0, 1]$ 사이이므로
 $|\lambda| \leq 1$ 이 성립하여 $\lambda = 1$ 이 최대 고유값이다

정상 상태(Steady State)와 거듭제곱법

$$\mathbf{v}_{k+1} = M\mathbf{v}_k$$

이 과정을 반복하다 보면 어느 순간 $\mathbf{v}_{k+1} = \mathbf{v}_k$

더 이상 바뀌지 않는 정상 상태 $\mathbf{v}^*$ 에 도달한다

이를 수식으로 나타내면

$$M\mathbf{v}^* = \mathbf{v}^*$$

정상 상태(Steady State)와 거듭제곱법

이는 고유벡터의 정의에 해당하는 부분과 같다

이 벡터가 PageRank 벡터다

즉 PageRank는 점수를 재분배해도 순위가 변하지 않는 안정 상태이고

그 답이 고유값 1의 고유벡터다

정상 상태(Steady State)와 거듭제곱법

특성방정식을 풀어 고유벡터를 직접 구하는 방법은 수많은 페이지로 이루어진 행렬에서 현실적으로 불가능하다

대신 거듭제곱법을 쓴다

M 이 대각화 가능하면 $M = PDP^{-1}$ 이고

$$M^k \mathbf{v}_0 = PD^k P^{-1} \mathbf{v}_0$$

정상 상태(Steady State)와 거듭제곱법

초기 벡터 $\mathbf{v}_0$ 를 고유벡터들의 선형결합으로 표현하면

$$M^k \mathbf{v}_0 = c_1 \lambda_1^k \mathbf{p}_1 + c_2 \lambda_2^k \mathbf{p}_2 + \cdots + c_n \lambda_n^k \mathbf{p}_n$$

$\lambda_1 = 1$ 이 최대 고유값이고 나머지는 $|\lambda_i| < 1$ 이므로 $k \rightarrow \infty$ 일 때

$$M^k \mathbf{v}_0 \rightarrow c_1 \mathbf{p}_1 = \mathbf{v}^*$$

정상 상태(Steady State)와 거듭제곱법

행렬을 반복해서 곱하는 것만으로 자동으로 고유벡터가 추출된다

수렴 속도는 $|\lambda_2/\lambda_1|$ 비율에 의존하며

두 번째 고유값이 1보다 충분히 작을수록 빠르게 수렴한다.

Dead End 문제

앞서 예시로 행렬을 구성했다

$$M = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0.5 & 0 & 0 & 0 \\ 0.5 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Dead End 문제

현재 나가는 링크가 없는 페이지(D)의 열 합이 0이다

D가 받은 중요도가 어디에도 전달되지 않아 확률 질량(probability mass)이 소멸한다

벡터의 원소 합이 매 단계마다 줄어들어 결국 0에 수렴한다

Dead End 문제

현재 나가는 링크가 없는 페이지(D)의 열 합이 0이다

D가 받은 중요도가 어디에도 전달되지 않아 확률 질량(probability mass)이 소멸한다

벡터의 원소 합이 매 단계마다 줄어들어 결국 0에 수렴한다

선형대수적으로 보면 D의 열이 0이면 M 은 확률 행렬 조건(열의 합이 1)을 만족하지 않는다

따라서 고유값 1이 최대임을 보장하는 앞의 증명이 성립하지 않는다

Dead End 문제

해결책

Langville & Meyer(2004) – Dead End 열의 원소를 모두 $1/n$ 으로 보정한다

Dead End에 도달한 사용자가 무작위 페이지로 이동한다고 해석한다

보정 후 열 원소들의 합이 모두 1을 만족해 확률 행렬의 조건이 복원된다

$$M' = \begin{pmatrix} 0 & 0 & 1 & 0.25 \\ 0.5 & 0 & 0 & 0.25 \\ 0.5 & 1 & 0 & 0.25 \\ 0 & 0 & 0 & 0.25 \end{pmatrix}$$

Dead End 문제

Langville과 Meyer는 Page와 Brin의 논문의 해결 방법이었던 Dead End의 열을 제거한 뒤 수렴 후 다시 추가하는 방식이 맞지 않다고 지적하며 이러한 방법을 설계했다

Spider Trap 문제

Dead End와는 독립적으로 다른 문제가 발생할 수 있다

Spider Trap이라고 한다

Spider Trap 문제

서로만 링크를 거는 페이지 집합이 있으면 모든 점수가 그 집합에만 분배된다

이 경우 M' (앞서 Dead End 해결한 행렬)은 확률 행렬 조건을 만족하지만
0인 원소가 존재하기 때문에 고유벡터가 유일하지 않을 수 있다

거듭제곱법의 수렴 결과가 초기 벡터인 $\mathbf{v}_0$ 에 따라 달라진다.

Spider Trap 문제

수학적으로 이 문제를 해결하려면 더 강한 조건이 필요하다

Spider Trap 문제

수학적으로 이 문제를 해결하려면 더 강한 조건이 필요하다

그것이 바로 Perron–Frobenius 정리

Spider Trap 문제

수학적으로 이 문제를 해결하려면 더 강한 조건이 필요하다

그것이 바로 Perron–Frobenius 정리

모든 원소가 양수인 행렬은 고유값 1이 중복 없이 유일하고

그에 대응하는 양수 고유벡터가 유일하게 존재한다

Spider Trap 문제

이 정리를 활용하기 위해 M' 에 조작을 가한다

$$G = d \cdot M' + \frac{1-d}{n} \mathbf{1} \times \mathbf{1}^T, \quad d = 0.85$$

G 의 모든 원소는 최소 $\frac{1-d}{n} = \frac{0.15}{n} > 0$ 이므로 양수이다

Spider Trap 문제

이는 사용자가 15%의 확률로 현재 페이지와 무관하게 임의의 페이지로 이동한다는 확률적 해석이기도 하다

그리고 이것을 댐핑 인자를 적용한다 한다

결론 - 구글 행렬

Dead End 보정($M \rightarrow M'$)과 댐핑 팩터($M' \rightarrow G$)를 순서대로 적용하면
최종적으로 구글 행렬(Google Matrix) G 가 완성된다

결론 - 구글 행렬

Dead End 보정($M \rightarrow M'$)과 댐핑 팩터($M' \rightarrow G$)를 순서대로 적용하면

최종적으로 구글 행렬(Google Matrix) G 가 완성된다

열의 원소 합이 1을 만족해 확률 행렬 조건을 만족하고

결론 - 구글 행렬

Dead End 보정($M \rightarrow M'$)과 댐핑 팩터($M' \rightarrow G$)를 순서대로 적용하면

최종적으로 구글 행렬(Google Matrix) G 가 완성된다

열의 원소 합이 1을 만족해 확률 행렬 조건을 만족하고

모든 원소가 양수이기 때문에 Perron–Frobenius 정리의 조건을 만족한다

컬론 - 구글 행렬

따라서 Perron–Frobenius 정리에 의해

G 의 고유값 1은 중복 없이 유일하다

대응하는 고유벡터(PageRank 벡터)는 유일하다

어떤 초기 벡터 $\mathbf{v}_0$ 에서 시작해도 반드시 이 고유벡터로 수렴한다

구현 코드(GitHub)와 수렴 판정

아래 내용은 이 탐구를 바탕으로 직접 설계하고 제작했다

구현 코드(GitHub)와 수렴 판정

실제 구현에서는 $n \times n$ 행렬 G 전체를 저장하지 않고

업데이트 식을 전개해서 사용한다

$\mathbf{v}_k$ 의 원소 합이 1이면 $\mathbf{1}\mathbf{1}^T \mathbf{v}_k = 1$ 이므로

$$\mathbf{v}_{k+1} = G\mathbf{v}_k = d \cdot M'\mathbf{v}_k + \frac{1-d}{N}\mathbf{1}$$

G 를 명시적으로 구성하지 않아도 같은 결과를 얻는다

구현 코드(GitHub)와 수렴 판정

참고한 기존 논문들은 50회에서 100회의 고정 반복을 사용했다

이번 구현에서는 두 연속 벡터의 L1 노름 기반 수렴 판정을 도입했다

$$\|\mathbf{v}_{k+1} - \mathbf{v}_k\|_1 = \sum_{i=1}^N |v_{k+1,i} - v_{k,i}| < \epsilon$$

이 방식은 수렴이 이미 이루어진 뒤의 불필요한 반복을 생략하고

실제 수렴 횟수를 측정할 수 있어 알고리즘 비교 분석이 가능하다.

구현 코드(GitHub)와 수렴 판정

기존 논문의 모호한 Dead End 처리를 확률 행렬 조건 복원 관점에서 조정하고
고정 반복 대신 노름 기반 수렴 판정을 도입해 알고리즘의 엄밀함을 높였다

구현 코드(GitHub)와 수렴 판정

구현한 GitHub 주소

<https://github.com/TETO-IS-KAWAI/PageRankCustom>

코드를 작성한 후 분석해 전개되는 내용은 이 저장소의 README에 있습니다

<https://github.com/TETO-IS-KAWAI/PageRankCustom/blob/main/README.md>

THANK
YOU